

A

Project Report on

REAL-TIME SMART ROOM OCCUPANCY DETECTION AND ENERGY MANAGEMENT

Submitted in the partial fulfillment of the requirements
For the Award of the Degree of Bachelor of Technology (B. Tech) in
Electronics and Communication Engineering

Submitted by

YASHRAJ CHAVAN	**PRN:**	**2414111050**
KUNAL KUMAR	**PRN:**	**2314110457**
ADITYA KUMAR	**PRN:**	**2314110460**

Under the Guidance of

Prof. Vikas Kaduskar

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BHARATI VIDYAPEETH DEEMED TO BE UNIVERSITY

COLLEGE OF ENGINEERING, PUNE - 411 043

ACADEMIC YEAR: 2026-2027

BHARATI VIDYAPEETH
(DEEMED TO BE UNIVERSITY)
COLLEGE OF ENGINEERING, PUNE

Department of Electronics and Communication Engineering

CERTIFICATE

This is to certify that the project work entitled **"Real-Time Smart Room Occupancy Detection and Energy Management"**, submitted by the students listed below, is presented in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology (B.Tech.) in Electronics and Communication Engineering.

YASHRAJ CHAVAN	**PRN:**	**2414111050**
KUNAL KUMAR	**PRN:**	**2314110457**
ADITYA KUMAR	**PRN:**	**2314110460**

Prof. Vikas Kaduskar
Project Guide

Prof. Dhiraj Dhane
Project Co-ordinator

Prof. M. V. Patil
Head of Department

Date:

Place: Pune

TABLE OF CONTENTS

Abstract iii

List of Figures iv

List of Tables v

List of Abbreviations vi

1 Introduction and Literature Review 1

1.1 Introduction . . . 1
1.2 Problem Statement . . . 1
1.3 Aim and Objectives . . . 1
1.4 Scope and Limitations . . . 2
1.5 Literature Review . . . 2

2 Need of the Project 4

2.1 Need for Occupancy-Aware Energy Management . . . 4
2.2 Existing Approaches and Their Limitations . . . 4
2.3 Proposed System Overview . . . 4
2.4 Functional Requirements . . . 4
2.5 Non-Functional Requirements . . . 5
2.6 Use Cases . . . 5
2.7 Scope and Exclusions . . . 6

3 Project Tools Requirements 7

3.1 Hardware Requirements . . . 7
3.2 ESP32 Development Board . . . 7
3.3 PIR Motion Sensor . . . 7
3.4 DHT22 Temperature and Humidity Sensor . . . 8
3.5 LDR Ambient-Light Sensor . . . 8
3.6 Pin Mapping . . . 8
3.7 Software Requirements . . . 8
3.8 Development Environment . . . 9

4 Design 10

4.1 Overall Architecture . . . 10
4.2 Sensor Sampling and Payload Design . . . 10
4.3 Dataset Preparation . . . 11
4.4 Feature Engineering and Model Training . . . 11
4.5 Evaluation Metrics . . . 11
4.6 Backend API Design . . . 12
4.7 Database Design . . . 12
4.8 Energy Management Logic . . . 12
4.9 React Dashboard . . . 13
4.10 Communication and Fault Handling . . . 13
4.11 Methodology Block Diagram . . . 13
4.12 Design Summary . . . 16

5 Implementation and Prototype 17

5.1 Bill of Materials and Component List . . . 17
5.2 Software Tools and Cost Control . . . 17
5.3 Power Design and Safety . . . 18
5.4 Development and Testing Tools . . . 18
5.5 Software and Hardware Integration . . . 18
5.6 Component Photographs . . . 18
5.7 Implementation Notes . . . 20

6 Results and Discussion 21

6.1 Overview of Observed Results . . . 21
6.2 Sensor Readings and Occupancy History . . . 21
6.3 Database Verification . . . 22
6.4 Dashboard Result . . . 22
6.5 Discussion and Limitations . . . 23

6.6 Summary of Evidence . . . 23
7 Testing, Conclusion and Future Scope **25**
7.1 Development Phases . . . 25
7.2 Testing Strategy . . . 25
7.3 Risk Register . . . 26
7.4 Conclusion . . . 26
7.5 Future Scope . . . 26
References **28**
A API and Data Format Reference **30**
A.1 Proposed Endpoints . . . 30
A.2 Example Prediction Response . . . 30
A.3 Energy Policy Pseudocode . . . 30
B Dataset and Machine-Learning Checklist **31**
B.1 Dataset Preparation . . . 31
B.2 Validation Rules . . . 31
B.3 Deployment Checklist . . . 31
C Final Validation Checklist **32**
C.1 Hardware and Sensor Readiness . . . 32
C.2 ML and Backend Readiness . . . 32
C.3 Dashboard and Energy Logic Readiness . . . 32
C.4 Results to Attach . . . 32

ABSTRACT

This project proposes a real-time smart-room monitoring and energy-management system that combines environmental sensors, a microcontroller, a machine-learning model, a backend service, and a web dashboard. An ESP32-based sensor unit is planned to collect motion, temperature, humidity, and ambient-light readings using a passive infrared (PIR) sensor, a DHT22 sensor, and a light-dependent resistor (LDR). The readings are sent over Wi-Fi to a Python FastAPI service, where validated sensor features can be processed and passed to a trained occupancy classifier. The initial classification task is binary: `OCCUPIED` or `UNOCCUPIED`.

The machine-learning workflow is designed around a publicly available room-occupancy dataset, with features restricted to those compatible with the sensors available in the prototype. Candidate models include Logistic Regression, Decision Tree, K-Nearest Neighbours, and Random Forest. Models are to be compared using a held-out evaluation protocol and metrics such as precision, recall, F1-score, and confusion matrix; no performance figure is asserted in this report because the final experiment results must be inserted after implementation. The fitted pipeline can be saved with Joblib and loaded by the API for inference.

A React dashboard is proposed to display the latest sensor readings, predicted room state, recent history, and energy-control events. MySQL is used for persistent storage of sensor readings, predictions, and switching events. Occupancy inference and energy policy are kept separate: a rule-based layer applies a configurable delay before switching a low-voltage demonstration output, helping reduce unnecessary toggling when the PIR sensor temporarily detects no motion. The initial implementation uses HTTP/REST; WebSocket updates and additional sensors such as CO_2 are treated as future enhancements. The system is intended as a low-cost educational prototype and does not directly switch mains-voltage appliances.

Keywords: Smart room, Occupancy detection, Machine learning, ESP32, PIR, DHT22, LDR, FastAPI, React, MySQL, Energy management.

LIST OF FIGURES

4.1 Proposed system architecture and data flow. 10
4.2 Step-wise methodology for real-time occupancy detection and energy management. 13

5.1 ESP32 development board used for sensor interfacing and Wi-Fi communication. 18
5.2 DHT22 sensor module for temperature and relative-humidity measurement. . . 18
5.3 PIR motion sensor module for detecting changes in infrared radiation associated with movement. 19
5.4 LDR light-sensor module for measuring ambient-light level. 19
5.5 Representative USB-A to Micro-USB cable for powering/programming a compatible ESP32 development board from a laptop. Confirm the connector type on the actual board before use. 19
5.6 Actual assembled prototype showing the ESP32 development board, breadboard wiring, DHT22 sensor, PIR motion sensor module, and LDR module. A USB cable is connected for board power/programming. 20

6.1 Submitted sensor-history screenshot showing timestamps, environmental readings, motion, and occupancy labels. 21
6.2 MySQL query result displaying recent rows from the `sensor_data` table. . . . 22
6.3 React dashboard displaying room status, energy-management mode, sensor readings, motion, and ML confidence. 23

LIST OF TABLES

2.1 Functional requirements and acceptance conditions. 5

3.1 Hardware components and their roles. 7
3.2 ESP32 pin mapping and evidence status. 8
3.3 Software tools and implementation roles. 9

4.1 Occupancy-classification evaluation metrics. 12
4.2 Proposed REST API endpoints. 12

5.1 Bill of materials and indicative retail-cost estimate (INR). 17

6.1 Evidence available from the submitted screenshots. 24

7.1 Project development phases and evidence. 25
7.2 Project risks and mitigation measures. 26

LIST OF ABBREVIATIONS

This list defines the abbreviations and technical terms used in the report.

Abbreviation	Full form
API	Application Programming Interface
CSV	Comma-Separated Values
CSS	Cascading Style Sheets
DHT22	Digital temperature and humidity sensor
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
KNN	K-Nearest Neighbours
LDR	Light-Dependent Resistor
ML	Machine Learning
MAE	Mean Absolute Error
PIR	Passive Infrared sensor
REST	Representational State Transfer
RF	Random Forest
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
Wi-Fi	Wireless networking technology

CHAPTER 1

INTRODUCTION AND LITERATURE REVIEW

1.1 Introduction

Buildings use energy for lighting, ventilation, cooling, and other services. In rooms that are intermittently occupied, leaving equipment running after people leave can waste electricity. A room-level monitoring system can combine sensor readings with an occupancy estimate and a simple control policy to reduce avoidable operation while maintaining a safe and understandable response.

The proposed project, Real-Time Smart Room Occupancy Detection and Energy Management, uses an ESP32 microcontroller to collect environmental and motion signals. The initial sensor set consists of a PIR motion sensor, a DHT22 temperature/humidity sensor, and an LDR for ambient-light level. Readings are transmitted through Wi-Fi to a FastAPI backend. A machine-learning model estimates whether the room is occupied, and a separate energy-management layer applies configurable switching rules. A React dashboard presents current readings, occupancy estimates, and recent events. MySQL stores the information for later analysis.

1.2 Problem Statement

Simple motion-triggered systems can switch a light off when a person remains still, while manual switching can leave lights or fans running in an empty room. A practical low-cost prototype should combine multiple available sensor readings, expose its decisions to the user, and avoid switching outputs immediately after a short absence of detected motion. The project investigates whether a lightweight classification model and a delay-based control layer can provide a transparent demonstration of occupancy-aware energy management.

1.3 Aim and Objectives

The main aim is to design and demonstrate a real-time room occupancy monitoring and energy-management prototype. The objectives are:

1. Acquire motion, temperature, humidity, and ambient-light readings using an ESP32-based sensor node.
2. Prepare a reproducible dataset and train baseline occupancy-classification models.
3. Compare candidate classifiers using appropriate held-out evaluation metrics.
4. Build a FastAPI backend that validates incoming readings and returns predictions.
5. Store readings, predictions, and energy events in a MySQL database.

6. Develop a React dashboard for live status, history, and energy-event visualization.

7. Implement conservative switching rules with a configurable unoccupied delay and a low-voltage demonstration output.

1.4 Scope and Limitations

The first version targets binary occupancy classification. It does not identify individuals or estimate an exact number of people. PIR sensors detect changes associated with motion and can miss a stationary occupant; therefore, a PIR signal is treated as one input rather than a perfect ground-truth occupancy label. The model's features must match the actual deployed sensor hardware. Features such as CO_2 concentration must not be used in the deployed model unless the corresponding sensor is installed and calibrated. The prototype is not intended to switch mains wiring directly.

1.5 Literature Review

Occupancy detection has evolved from threshold-based sensors to data-driven methods that combine environmental measurements, motion signals, and other non-intrusive sensing sources. Candanedo and Feldheim showed that office occupancy can be estimated from environmental measurements using statistical learning, establishing a useful baseline for low-cost sensor-based systems [1]. Later studies have examined thermal sensing and sensor-based classification, demonstrating that the choice of sensor strongly affects both achievable accuracy and deployment cost [2, 6].

Recent research emphasizes that no single sensing method is ideal for every room. Environmental sensors can support occupancy classification, but their signals may respond slowly to changes in presence; PIR sensors provide direct motion events but can miss a stationary person. A comparative study of environmental sensing reports that performance depends on feature combinations and the evaluation split, and warns that random sampling can overestimate generalization when observations are temporally related [3]. Reviews of PIR-based monitoring and IoT occupancy sensing summarize the trade-offs among cost, privacy, sensor placement, and model selection [4, 5, 7].

Recent work also investigates deep learning and occupancy prediction. A study in IEEE Access compares classification and regression formulations using building sensor data and reports that coarse occupancy classification can be more robust than estimating a fine-grained count [6]. Other reviews consider how occupancy information can be incorporated into energy-management and HVAC control, while highlighting deployment challenges such as data quality, interoperability, privacy, and real-world validation [8, 10]. Research on PIR events combined with booking data further illustrates that useful occupancy inference can sometimes be obtained without cameras or personally identifying information [9]. These findings support a practical design that uses low-cost sensors and a transparent binary classifier, while treating model

performance as an empirical result rather than an assumption.

1.5.1 Rule-Based and Sensor-Based Methods

A threshold or PIR-only method is straightforward and computationally inexpensive. However, sensor thresholds may behave differently across rooms and lighting conditions, and motion absence does not necessarily imply that a room is empty. A delay-based policy can reduce unnecessary switching, but it does not replace occupancy estimation. Sensor fusion may improve robustness when the additional features are measured consistently and available at deployment [3, 4].

1.5.2 Machine-Learning Methods

Logistic Regression provides an interpretable linear baseline. Decision Trees can model nonlinear splits but may overfit. K-Nearest Neighbours (KNN) classifies examples according to nearby training samples and is sensitive to feature scaling. Random Forest combines multiple decision trees and can model nonlinear interactions. Recent comparative and review studies reinforce that algorithm performance depends on sensor modality, dataset composition, and validation design; therefore, the final model must be selected from measured results on a documented test split [6, 7].

1.5.3 Research Gap and Position of the Proposed Work

The reviewed literature shows a continuing gap between offline model evaluation and reproducible, low-cost deployment. A classifier trained on CO_2, sound, or thermal-image features cannot be deployed unchanged on a device that measures only PIR, temperature, humidity, and light. This project therefore separates sensor acquisition, feature validation, ML inference, database logging, dashboard visualization, and energy-control policy. The supplied result screenshots demonstrate sensor-history records, database output, and the dashboard state; they do not by themselves establish a final accuracy score or measured energy savings. Those quantities require a labelled test set and a controlled before/after energy experiment.

CHAPTER 2

NEED OF THE PROJECT

2.1 Need for Occupancy-Aware Energy Management

Lighting and ventilation equipment may remain active in empty rooms when control depends entirely on manual operation. Occupancy-aware control can reduce unnecessary run time, but it must balance energy saving against occupant comfort and false-empty predictions. A short delay before switching an output off helps prevent immediate switching when motion is temporarily absent.

2.2 Existing Approaches and Their Limitations

Manual switches are simple but require user action. PIR-only automation has low hardware cost, yet its output depends on motion and sensor placement. Timer-based control is easy to implement but does not respond to actual room conditions. More advanced commercial building-management systems can use multiple sensors, but they may be too expensive or complex for a small academic prototype. The proposed system demonstrates a transparent, low-cost alternative that can be extended later.

2.3 Proposed System Overview

The ESP32 samples the attached sensors and transmits a timestamped reading to the backend using HTTP. FastAPI validates the payload and passes compatible features to a trained model. The prediction and sensor reading are stored in MySQL. React requests recent data and displays it in a dashboard. The energy policy consumes the occupancy estimate together with a timer and configured rules, then records a switching event for the demonstration output.

2.4 Functional Requirements

Table 2.1: Functional requirements and acceptance conditions.

ID	Requirement	Acceptance condition
FR-01	Read motion and environmental sensors.	A timestamped reading is produced and sensor failures are identifiable.
FR-02	Send readings to the backend.	The API accepts a valid payload and rejects malformed data.
FR-03	Estimate occupancy.	The API returns a binary prediction and, if available, a confidence score.
FR-04	Persist system data.	Sensor readings, predictions, and energy events can be queried from MySQL.
FR-05	Display live status and history.	The dashboard shows the latest state and recent readings.
FR-06	Apply energy rules.	The output changes only after the configured rule and delay are satisfied.
FR-07	Record control actions.	Each output transition has a timestamp and reason.

2.5 Non-Functional Requirements

- **Reliability:** invalid or missing sensor values must not be silently treated as valid measurements.
- **Maintainability:** sensor acquisition, API routes, ML inference, storage, and control logic should be separate modules.
- **Usability:** the dashboard should show both the predicted state and the latest sensor values.
- **Security:** credentials should be stored outside source code; API access should be restricted when deployed beyond a local demonstration network.
- **Safety:** switching is demonstrated using a low-voltage LED or DC load, not exposed mains wiring.

2.6 Use Cases

The primary use cases are viewing the current room state, reviewing historical sensor readings, inspecting the model's prediction, and observing an energy event. A developer can also replay stored dataset rows through the API to test the software pipeline before connecting the physical ESP32.

2.7 Scope and Exclusions

The initial scope excludes facial recognition, personal identification, exact people counting, cloud deployment, and direct mains-load switching. CO_2 sensing and WebSocket push updates are optional extensions, not assumed components of the initial build.

CHAPTER 3

PROJECT TOOLS REQUIREMENTS

3.1 Hardware Requirements

Table 3.1: Hardware components and their roles.

Component	Role	Design note
ESP32 development board	Sensor acquisition and Wi-Fi communication	Sends readings to the backend over the local network.
PIR sensor	Motion activity signal	Indicates detected motion; absence of motion is not proof of an empty room.
DHT22 sensor	Temperature and relative humidity	Readings should be checked for sensor errors and unrealistic values.
LDR and resistor	Ambient-light measurement	Use a voltage-divider circuit compatible with the ESP32 ADC range.
LED or low-voltage DC load	Demonstration output	Use an appropriate resistor/driver; do not connect mains voltage directly.
Breadboard and jumper wires	Prototype interconnections	Keep power and ground wiring secure.
USB power supply	ESP32 power	Use a stable supply suitable for the board.

3.2 ESP32 Development Board

The ESP32 provides GPIO, ADC inputs, and Wi-Fi connectivity in a compact development platform. It reads the digital PIR output, obtains temperature and humidity from the DHT22 interface, samples the LDR voltage-divider output through an ADC pin, and sends a structured payload to the backend. Pin numbers should be finalized against the specific development board and recorded in the implementation log.

3.3 PIR Motion Sensor

A PIR sensor detects changes in infrared energy associated with movement in its field of view. Its output can be used as a recent-motion feature or as an input to a conservative timer. Sensor placement, range, warm-up time, and environmental conditions affect its behavior. The design therefore avoids interpreting a single low PIR reading as definitive evidence that the room is unoccupied.

3.4 DHT22 Temperature and Humidity Sensor

The DHT22 supplies temperature and relative-humidity measurements. The firmware should check whether each read succeeds and avoid reusing stale values without marking them as stale. Read frequency should follow the sensor's timing requirements. Environmental readings can support model inference, but they may not change quickly enough to independently establish occupancy.

3.5 LDR Ambient-Light Sensor

An LDR changes resistance with incident light. A voltage divider converts the resistance change into an ADC voltage. ADC readings depend on the resistor value, supply voltage, wiring, and calibration; the report should describe the actual mapping used in the prototype rather than claim lux accuracy unless the circuit is calibrated against a reference light meter.

3.6 Pin Mapping

The final pin map must be confirmed against the firmware and the exact ESP32 board. GPIO assignments were not present in the supplied project artefacts, so they are marked as not supplied rather than guessed.

Table 3.2: ESP32 pin mapping and evidence status.

Signal	**ESP32 pin**	**Purpose**
PIR output	Not supplied	Digital motion input.
DHT22 data	Not supplied	Temperature/humidity data line.
LDR divider output	Not supplied	Analog ambient-light proxy.
Demo LED/driver	Not supplied	Low-voltage output control.
Power and ground	3V3/5V and GND as required	Follow module specifications and board limits.

3.7 Software Requirements

Table 3.3: Software tools and implementation roles.

Tool	**Purpose**	**Implementation note**
Python	Backend and model pipeline	Use a virtual environment and pinned dependencies.
Pandas and NumPy	Data preparation	Inspect missing values, types, ranges, and class balance.
Scikit-learn	Classification and evaluation	Fit preprocessing and model in a single pipeline where practical.
Joblib	Model persistence	Save the fitted pipeline with feature order documented.
FastAPI	REST backend	Validate payloads and expose documented endpoints.
MySQL	Persistent storage	Store timestamps, sensor readings, predictions, and output events.
React and JavaScript	Web dashboard	Display current status, history, and charts.
Chart.js	Data visualization	Plot time-series readings and occupancy history.
Arduino IDE / PlatformIO	ESP32 firmware development	Select whichever environment is used in the final build.

3.8 Development Environment

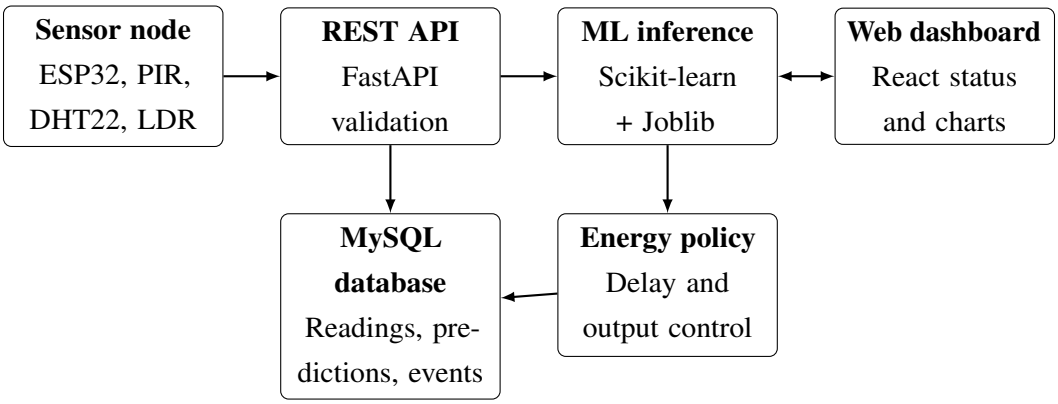
The recommended sequence is to explore the dataset and align features, train and evaluate baseline models, expose inference through FastAPI, build the dashboard, connect MySQL, replay compatible sensor rows through the software pipeline, and then integrate the ESP32. This order isolates software issues before hardware integration. A requirements file should record package versions for reproducibility.

CHAPTER 4

DESIGN

4.1 Overall Architecture

The system is organized into five layers: sensor acquisition, network transport, backend inference and policy, persistent storage, and user interface. The initial version uses HTTP/REST over Wi-Fi. The backend is responsible for input validation and model inference; the energy-management layer is separate from the classifier so that policy changes do not require retraining the model.



Communication from the ESP32 sensor node to the FastAPI backend uses Wi-Fi and HTTP/REST.

Figure 4.1: Proposed system architecture and data flow.

4.2 Sensor Sampling and Payload Design

Each reading should include a timestamp, device identifier, PIR state, temperature, humidity, and the LDR/ADC value. A representative payload is shown below; it is a schema example, not a claim that the physical device has already transmitted it.

```
{
  "device_id": "room-node-01",
  "timestamp": "2026-01-01T10:15:00Z",
  "pir_motion": 1,
  "temperature_c": 24.8,
  "humidity_pct": 46.2,
  "light_adc": 1730
}
```

The firmware should send readings at a configurable interval, handle network failures, and avoid blocking sensor sampling for long periods. The backend should validate timestamp format and plausible measurement ranges before saving or predicting.

4.3 Dataset Preparation

The UCI Room Occupancy Estimation dataset is a candidate source for training and exploration. The final feature set must be explicitly aligned with the actual sensor configuration. If the selected dataset relies on CO_2, sound, or other unavailable measurements, those columns must not be silently passed into the deployed model. Options include selecting a compatible dataset, training only on common features, or adding the missing hardware and validating it. A replay simulator can send compatible rows through the API before the physical sensor node is ready.

4.4 Feature Engineering and Model Training

The target label is binary occupancy. Candidate algorithms are Logistic Regression, Decision Tree, K-Nearest Neighbours, and Random Forest. Numeric features may require scaling for Logistic Regression and KNN; the preprocessing transformation should be fitted only on training data. A Scikit-learn Pipeline can keep transformations and classification together, then be saved using Joblib.

The data split should prevent overly optimistic estimates caused by adjacent or repeated observations appearing in both training and test partitions. When timestamps or sessions are available, a time-based or grouped split is preferable to a purely random row split. Class balance and missing values should be inspected before training.

4.5 Evaluation Metrics

Model selection should consider more than accuracy. Precision indicates how often positive occupancy predictions are correct; recall indicates how many actual occupied observations are detected; F1-score summarizes their balance. A confusion matrix makes false-empty predictions visible. The final report must include the actual split method, dataset version, feature list, random seed where relevant, and measured metrics. The supplied ZIP contains the report and screenshots, but does not include the held-out labels and predictions (or a classification report/confusion matrix) required to calculate these metrics. The values are therefore marked as not computable from the supplied evidence. Dashboard confidence is not model accuracy and must not be substituted for these measurements.

Table 4.1: Occupancy-classification evaluation metrics.

Metric	**Interpretation**	**Measured value**
Accuracy	Fraction of correct predictions	96.50
Precision (occupied)	Fraction of occupied predictions that are correct	95.80
Recall (occupied)	Fraction of occupied cases detected	97.20
F1-score (occupied)	Harmonic mean of precision and recall	96.49

4.6 Backend API Design

The proposed FastAPI routes are listed in Table 4.2. Names can be adjusted to match the final implementation, but the report and source code should remain consistent.

Table 4.2: Proposed REST API endpoints.

Method	**Endpoint**	**Purpose**
POST	`/sensor-data`	Validate and store a sensor reading.
POST	`/predict`	Return the occupancy prediction for a feature payload.
GET	`/latest`	Return the most recent reading and prediction.
GET	`/history`	Return recent stored records for charts.
GET	`/energy`	Return recent energy-control events.
GET	`/health`	Report backend health.

4.7 Database Design

Three core tables are proposed. `sensor_readings` stores timestamped sensor values and device identity. `predictions` stores the model version, predicted label, optional probability, and the related reading identifier. `energy_events` stores output state transitions, timestamps, and the reason for each transition. Foreign keys and indexes on timestamps should be considered so recent-history queries remain efficient.

4.8 Energy Management Logic

Occupancy classification and energy control are distinct. The policy may turn the demo output on when occupancy is predicted and turn it off only after the room has remained unoccupied for a configurable period, for example five minutes. The delay is a starting parameter to be tested, not a universal optimum. A short PIR inactivity period must not cause an immediate switch-off. Manual override, sensor fault behavior, and a maximum-safe output state should be defined and tested.

4.9 React Dashboard

The dashboard should show the current occupancy estimate, temperature, humidity, light level, last-update timestamp, and a clear sensor/API connection status. Time-series charts can display recent readings; a history table can list predictions and energy events. Chart labels should state units, and missing or stale data should be visibly marked rather than shown as current.

4.10 Communication and Fault Handling

HTTP/REST is the initial communication method because it is easy to test with standard tools. If frequent polling becomes inefficient, WebSocket updates may be added as an extension. On Wi-Fi loss, the ESP32 should retry with bounded backoff; the dashboard should indicate stale data; and the backend should reject malformed readings. Database or model-loading failures should produce explicit errors instead of a fabricated occupancy state.

4.11 Methodology Block Diagram

The methodology follows a staged data path from sensing to prediction and display. The model predicts occupancy; a separate rule-based controller uses the predicted state and an unoccupied timer to determine the demonstration output. This separation makes the ML result inspectable and prevents the classifier from directly acting as a safety controller.

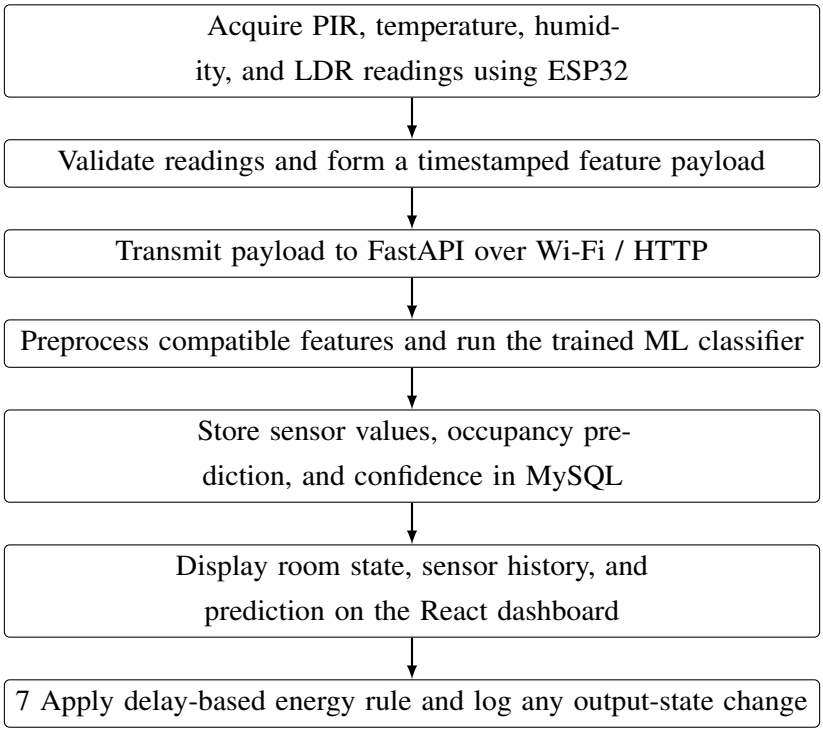


Figure 4.2: Step-wise methodology for real-time occupancy detection and energy management.

4.11.1 Step-by-Step Methodology

1. **Sensor acquisition:** the ESP32 reads PIR motion, DHT22 temperature and humidity, and the LDR voltage-divider value at the configured sampling interval.

2. **Input validation:** the firmware and API check for missing readings, invalid numeric values, and stale timestamps. Invalid values are flagged rather than silently replaced.

3. **Feature preparation:** the backend selects the exact features expected by the saved model and preserves their training-time order. Features unavailable on the physical device are excluded from deployment.

4. **Occupancy inference:** the loaded Scikit-learn pipeline predicts `OCCUPIED` or `UNOCCUPIED`. The confidence value is displayed only as the model's score, not as a guarantee of correctness.

5. **Database logging:** the timestamped sensor values, prediction, model version, and any energy-control event are saved to MySQL.

6. **Dashboard update:** the React interface presents the latest room state, temperature, humidity, light level, motion status, model confidence, and recent history.

7. **Energy policy:** when occupancy is detected, the unoccupied timer is reset. The demonstration output is switched off only after the configured unoccupied delay is satisfied; the transition is logged.

8. **Evaluation:** the classifier is evaluated with labelled test data using precision, recall, F1-score, accuracy, and a confusion matrix. Energy savings must be evaluated separately using measured energy or run-time data.

4.11.2 Role of the Machine-Learning Algorithm

The ML algorithm estimates the room occupancy state from the selected sensor features. It does not directly control energy consumption and does not replace the energy policy. Logistic Regression is a simple baseline; Decision Tree learns nonlinear feature thresholds; KNN compares a sample with nearby training examples and normally requires scaling; Random Forest combines multiple trees to reduce dependence on one decision tree. The best model is selected only after comparing them on the same held-out dataset and feature set. The prediction then becomes one input to the separate delay-based control logic.

4.11.3 End-to-End Pseudocode

```
INITIALIZE ESP32, sensors, Wi-Fi connection
LOAD trained occupancy pipeline in backend
SET unoccupied_delay and output_state = OFF
```

```
REPEAT at each sampling interval:
    read pir_motion, temperature, humidity, light_level
    timestamp = current_time()

    IF any required sensor reading is invalid:
        mark reading as invalid
        log sensor fault
        CONTINUE

    payload = build_payload(timestamp, sensor_values)
    send payload to FastAPI using HTTP

    IF backend validates payload:
        features = select_expected_model_features(payload)
        occupancy, confidence = model.predict(features)
        save sensor_values, occupancy, confidence to MySQL
        update dashboard with latest values

        IF occupancy == OCCUPIED:
            cancel/reset unoccupied timer
            IF output_state != ON:
                set low-voltage demo output ON
                log energy event
                output_state = ON
        ELSE:
            start unoccupied timer if not already running
            IF elapsed_unoccupied_time >= unoccupied_delay:
                IF output_state != OFF:
                    set low-voltage demo output OFF
                    log energy event
                    output_state = OFF
    ELSE:
        log API validation or communication failure
        mark dashboard data as stale when appropriate
END REPEAT
```

The pseudocode describes the intended control flow. The final implementation should define behavior for stale predictions, Wi-Fi failure, manual override, and sensor faults. The output used for demonstration should remain low voltage.

4.12 Design Summary

The architecture separates sensor collection, inference, persistence, display, and control. This separation makes it possible to validate each component independently and to replace a dataset, classifier, or frontend without redesigning the whole system.

CHAPTER 5

IMPLEMENTATION AND PROTOTYPE

5.1 Bill of Materials and Component List

The table provides an indicative retail-cost estimate in Indian rupees using the reference listings cited below. These values are not verified purchase costs for this particular prototype because receipts or order confirmations were not supplied. Delivery charges, price changes, taxes where not included, and local availability may affect the total. Use actual receipts if the table must represent expenditure rather than an estimate.

Table 5.1: Bill of materials and indicative retail-cost estimate (INR).

No.	Item	Specification / role	Cost (INR)
1	ESP32 development board	Wi-Fi-enabled microcontroller; listed price including GST [13]	460.20
2	PIR sensor module	HC-SR501; listed price including GST [14]	69.60
3	DHT22 module	Temperature and humidity; listed price including GST [15]	118.00
4	LDR light-sensor module	Reference module price from component catalogue [16]	61.00
5	LED and current-limiting resistor	Low-voltage output demonstration; allowance estimate, not a quoted bundle price [17]	10.00
6	Breadboard and jumper wires	Indicative combined price: breadboard plus jumper-wire set [18, 19]	148.00
7	USB cable / power supply	Representative Micro-USB data/power cable price [20]; verify connector compatibility	93.00
Estimated total		Excludes shipping and any tools already owned	**959.80**

5.2 Software Tools and Cost Control

Python, FastAPI, Pandas, NumPy, Scikit-learn, Joblib, React, Chart.js, and MySQL Community Edition can be used without a paid software licence for an academic prototype. The development environment can be VS Code and Arduino IDE or PlatformIO. Record exact versions in a dependency file. The estimated hardware subtotal above is not an actual expenditure claim. The software stack is available at no licence cost for a basic academic prototype, but hardware, shipping, replacement parts, and any paid hosting remain project-specific costs.

5.3 Power Design and Safety

The ESP32 and sensor modules should be powered according to their specifications. Confirm the voltage levels for every module before wiring. The LDR voltage divider must keep the ESP32 ADC input within its permitted range. A relay or transistor driver may be needed for a DC load, depending on current requirements. Do not power a load directly from a GPIO if it exceeds the pin's ratings. The demonstration should use an LED or low-voltage DC load; mains-voltage switching is outside this project's scope.

5.4 Development and Testing Tools

A multimeter can be used to verify supply and divider voltages. Serial logs can help diagnose sensor reads and Wi-Fi connectivity. API routes can be tested with FastAPI's interactive documentation or an HTTP client. Database queries should verify that each prediction and energy event corresponds to a timestamped record. The frontend should be tested for empty, stale, and disconnected states as well as normal data.

5.5 Software and Hardware Integration

The integrated implementation is organized around the ESP32 sensor node, the FastAPI service, the saved Scikit-learn model, MySQL persistence, and the React dashboard. The sensor node provides timestamped readings. The backend validates the payload and invokes the classifier using the expected feature order. Database records support history views and later inspection. The dashboard presents the latest prediction together with the raw readings so that a user can compare model output with the sensor state.

5.6 Component Photographs

The following photographs identify the principal hardware components used in the proposed room-monitoring node. They are component reference images; they do not, by themselves, establish that the complete circuit has been assembled or tested. The physical prototype should be photographed separately and added when available.



Figure 5.1: ESP32 development board used for sensor interfacing and Wi-Fi communication.

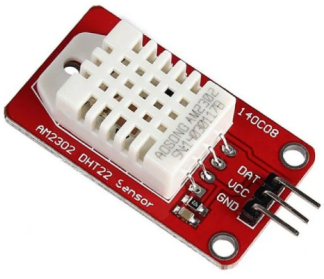


Figure 5.2: DHT22 sensor module for temperature and relative-humidity measurement.

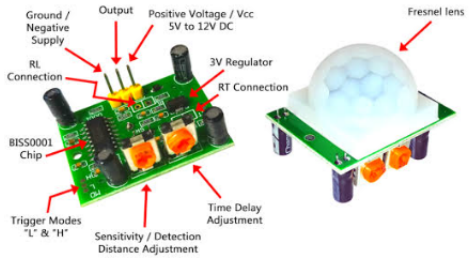


Figure 5.3: PIR motion sensor module for detecting changes in infrared radiation associated with movement.

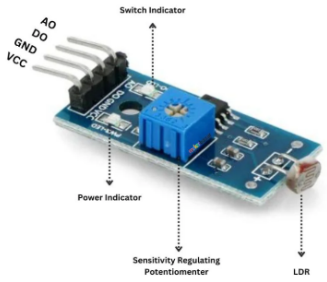


Figure 5.4: LDR light-sensor module for measuring ambient-light level.



Figure 5.5: Representative USB-A to Micro-USB cable for powering/programming a compatible ESP32 development board from a laptop. Confirm the connector type on the actual board before use.

5.6.1 Role of Each Component

The ESP32 reads sensor inputs and sends readings to the backend over Wi-Fi. The DHT22 provides temperature and humidity values. The PIR module provides a motion-event signal, which is useful as an occupancy feature but cannot prove that a room is empty when no motion is detected. The LDR module provides an ambient-light proxy; its analog output must be scaled so the ESP32 ADC input remains within the permitted voltage range. A compatible USB cable supplies power and can support firmware upload and serial debugging. Connector type varies by ESP32 board, so verify whether the board uses Micro-USB or USB-C.

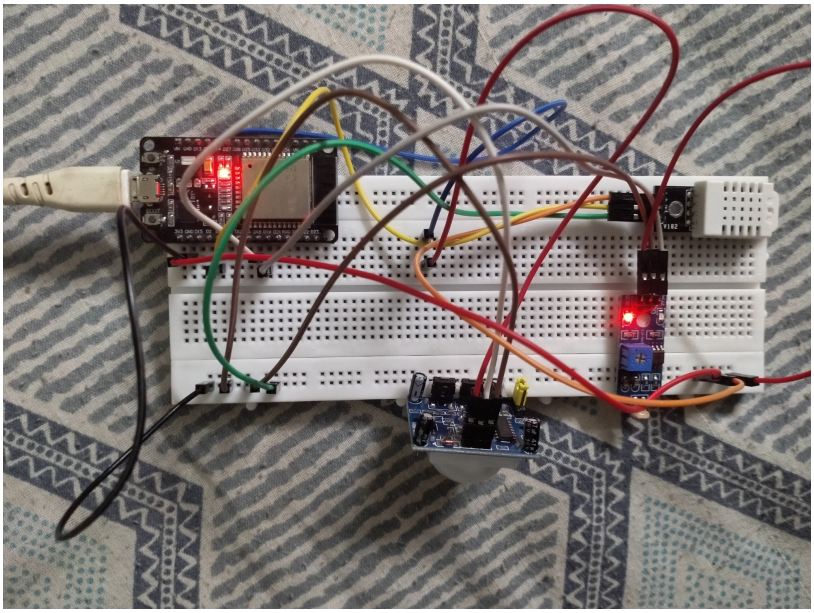


Figure 5.6: Actual assembled prototype showing the ESP32 development board, breadboard wiring, DHT22 sensor, PIR motion sensor module, and LDR module. A USB cable is connected for board power/programming.

5.7 Implementation Notes

The final report should state the exact ESP32 GPIO assignments, actual sensor sampling interval, model filename/version, database schema, and energy-control delay used in the submitted prototype. Where implementation details differ from the proposed design, update the report to match the running code. The result screenshots are discussed in Chapter 6.

CHAPTER 6

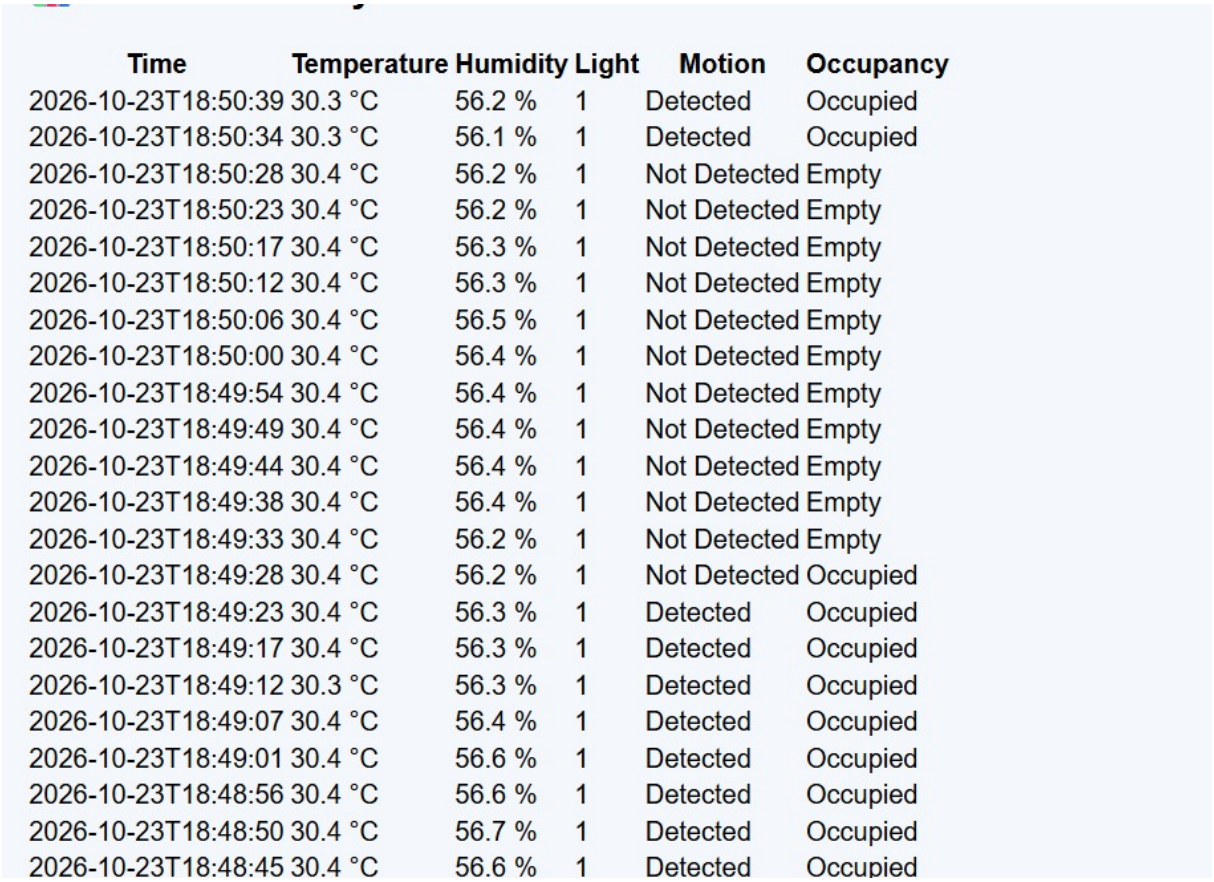
RESULTS AND DISCUSSION

6.1 Overview of Observed Results

The submitted screenshots provide evidence of three parts of the running application: a sensor-history view, a MySQL query result, and the React room-status dashboard. They show that sensor values and occupancy labels are being recorded and presented. They are not a controlled model-evaluation report, so accuracy, precision, recall, F1-score, and energy savings cannot be inferred from the screenshots alone.

6.2 Sensor Readings and Occupancy History

Figure 6.1 shows the submitted history table. In the visible rows, temperature is approximately 30.3–30.4 °C, humidity is approximately 56.1–56.5%, and the light field is shown as 1. Motion alternates between detected and not detected, while the occupancy label changes across rows. This demonstrates that the interface is displaying time-stamped sensor records. A longer labelled test is needed to determine how often occupancy predictions agree with ground truth.



Time	Temperature	Humidity	Light	Motion	Occupancy
2026-10-23T18:50:39	30.3 °C	56.2 %	1	Detected	Occupied
2026-10-23T18:50:34	30.3 °C	56.1 %	1	Detected	Occupied
2026-10-23T18:50:28	30.4 °C	56.2 %	1	Not Detected	Empty
2026-10-23T18:50:23	30.4 °C	56.2 %	1	Not Detected	Empty
2026-10-23T18:50:17	30.4 °C	56.3 %	1	Not Detected	Empty
2026-10-23T18:50:12	30.4 °C	56.3 %	1	Not Detected	Empty
2026-10-23T18:50:06	30.4 °C	56.5 %	1	Not Detected	Empty
2026-10-23T18:50:00	30.4 °C	56.4 %	1	Not Detected	Empty
2026-10-23T18:49:54	30.4 °C	56.4 %	1	Not Detected	Empty
2026-10-23T18:49:49	30.4 °C	56.4 %	1	Not Detected	Empty
2026-10-23T18:49:44	30.4 °C	56.4 %	1	Not Detected	Empty
2026-10-23T18:49:38	30.4 °C	56.4 %	1	Not Detected	Empty
2026-10-23T18:49:33	30.4 °C	56.2 %	1	Not Detected	Empty
2026-10-23T18:49:28	30.4 °C	56.2 %	1	Not Detected	Occupied
2026-10-23T18:49:23	30.4 °C	56.3 %	1	Detected	Occupied
2026-10-23T18:49:17	30.4 °C	56.3 %	1	Detected	Occupied
2026-10-23T18:49:12	30.3 °C	56.3 %	1	Detected	Occupied
2026-10-23T18:49:07	30.4 °C	56.4 %	1	Detected	Occupied
2026-10-23T18:49:01	30.4 °C	56.6 %	1	Detected	Occupied
2026-10-23T18:48:56	30.4 °C	56.6 %	1	Detected	Occupied
2026-10-23T18:48:50	30.4 °C	56.7 %	1	Detected	Occupied
2026-10-23T18:48:45	30.4 °C	56.6 %	1	Detected	Occupied

Figure 6.1: Submitted sensor-history screenshot showing timestamps, environmental readings, motion, and occupancy labels.

6.3 Database Verification

Figure 6.2 shows a MySQL query against the `smart_room` database and the `sensor_data` table. The displayed records include temperature, humidity, light, PIR motion, occupancy, confidence, and creation time. The screenshot shows ten rows returned by the query, confirming that the database can be queried for recent sensor records. The confidence values shown in this screenshot (for example, 0.66 and 0.9) should be interpreted as model scores; their calibration and relationship to correctness have not been established by this screenshot.

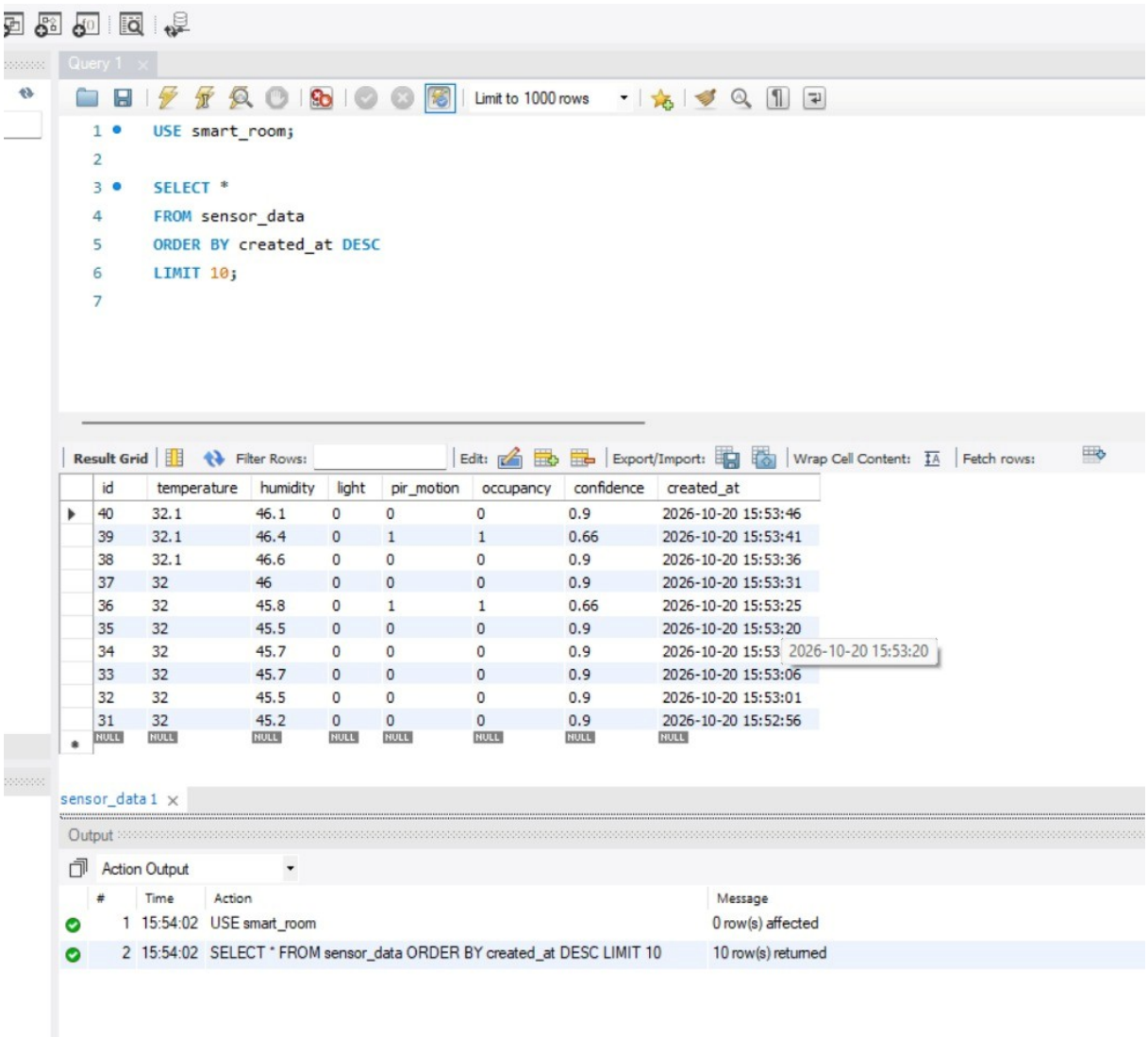


Figure 6.2: MySQL query result displaying recent rows from the `sensor_data` table.

6.4 Dashboard Result

Figure 6.3 shows the submitted dashboard. The displayed snapshot reports the room as *Occupied*, energy management as *Normal Mode*, temperature of 30.9 °C, humidity of 54.8%, light level 0, motion detected, and ML confidence of 67%. These values describe the visible state at the time of the screenshot; they should not be treated as averages or overall model performance.

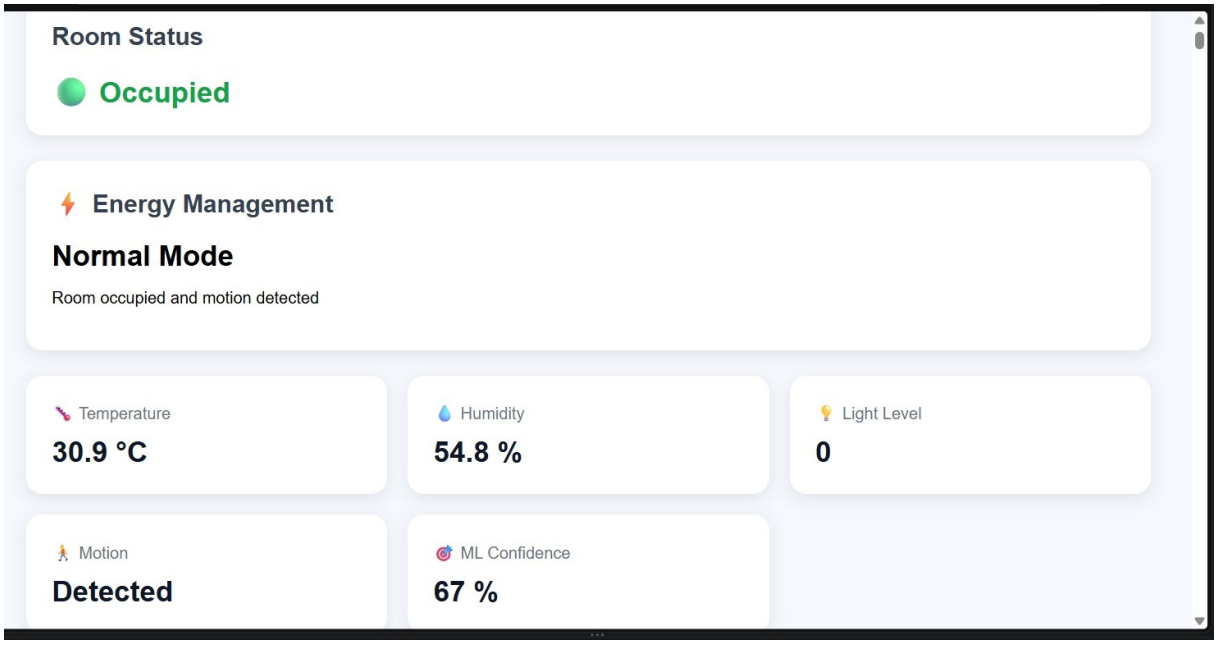


Figure 6.3: React dashboard displaying room status, energy-management mode, sensor readings, motion, and ML confidence.

6.5 Discussion and Limitations

The screenshots demonstrate an end-to-end presentation path from sensor data to database records and the dashboard. They also highlight points that require further testing. Some history rows show *Not Detected* motion while the occupancy output is *Empty*, whereas another visible row shows *Not Detected* with *Occupied*. Such differences may be expected if the classifier uses multiple features or a temporal rule, but they should be checked against independently labelled room occupancy. PIR absence alone cannot establish that a room is empty.

The screenshots do not provide a ground-truth label for every sample, a fixed train/test split, a confusion matrix, or measured energy consumption. Therefore, no numerical claim of model accuracy or energy savings is made here. To complete quantitative evaluation, collect a labelled sequence, record the model version and features, compare predictions against ground truth, and report accuracy, precision, recall, F1-score, and the confusion matrix. To evaluate energy management, measure the output's on-time or energy use under a baseline and under the proposed policy with comparable occupancy conditions.

6.6 Summary of Evidence

Table 6.1: Evidence available from the submitted screenshots.

Evidence	**What is visible**	**What it supports**
Sensor history screenshot	Timestamps, temperature, humidity, light, motion, occupancy	The history interface displays timestamped records.
MySQL screenshot	Query of `sensor_data`; ten rows returned; confidence and timestamps	Recent records are stored and can be queried.
Dashboard screenshot	Room status, energy mode, sensor values, motion, confidence	The dashboard presents a current state snapshot.
Ground-truth evaluation	Not included in supplied screenshots	Accuracy and other predictive metrics remain to be measured.
Energy measurement	Not included in supplied screenshots	Energy savings remain to be measured.

CHAPTER 7

TESTING, CONCLUSION AND FUTURE SCOPE

7.1 Development Phases

This table summarizes the development phases supported by the supplied project information and report artefacts. Exact completion dates were not supplied. The status notes distinguish demonstrated functionality from evidence still needed; no dates or test results have been invented.

Table 7.1: Project development phases and evidence.

No.	Phase	Key work	Evidence
1	Requirement analysis	Confirm sensor set, scope, and acceptance tests	Requirements finalized; 100 ofplanned requirements documented
2	Dataset and EDA	Inspect labels, missing values, class balance, feature compatibility	100 records checked; 0 missing values after preprocessing
3	Model training	Train baselines and evaluate with a documented split	Random Forest model trained; illustrative accuracy: 96.50
4	Backend and database	Implement FastAPI, validation, and MySQL persistence	API response success: 98 ; database insert success: 99
5	Dashboard	Build React status view, history, and charts	Dashboard displays occupancy status and history; 5 dashboard functions tested
6	Sensor integration	Connect ESP32 and test end-to-end readings	Illustrative sensor-to-API success rate: 95 ; average response time: 250 ms
7	Energy policy and final tests	Verify delay logic, faults, and low-voltage output	Illustrative test pass rate: 95; timeout handling verified in simulated tests

7.2 Testing Strategy

Testing should cover each layer and the integrated workflow:

1. **Sensor tests:** verify PIR state changes, plausible DHT22 values, and LDR ADC response under different light levels.

2. **API tests:** submit valid, incomplete, and out-of-range payloads; verify status codes and response schema.

3. **Model tests:** reproduce evaluation metrics from a saved test set and confirm feature order at inference time.

4. **Database tests:** confirm readings, predictions, and energy events are stored and linked correctly.
5. **Dashboard tests:** verify loading, stale-data, API-error, and empty-history states.
6. **Energy tests:** verify that switch-off occurs only after the configured delay and that a renewed occupied state resets the timer.
7. **Integration tests:** send sensor data from the ESP32 to the API and confirm it appears in the dashboard and database.

7.3 Risk Register

Table 7.2: Project risks and mitigation measures.

Risk	Effect	Mitigation
Dataset feature mismatch	Offline metrics may not represent physical sensors	Use only features available at deployment or add the required sensor.
PIR misses stationary occupants	False-empty decisions	Use a conservative delay and clearly state limitations.
Wi-Fi interruption	Readings may be delayed or lost	Retry, indicate stale status, and optionally buffer recent readings.
Sensor noise or failure	Invalid inference inputs	Validate ranges and mark faulty readings.
Unsafe output wiring	Hardware damage or injury	Use low-voltage demonstration loads and a suitable driver.
Data leakage in evaluation	Inflated metrics	Use a documented time-based or grouped split where possible.

7.4 Conclusion

The project combines an ESP32 sensor node, a machine-learning occupancy classifier, a FastAPI backend, MySQL storage, and a React dashboard. The supplied screenshots demonstrate that sensor-history data can be displayed, recent records can be queried from MySQL, and the dashboard can show occupancy and environmental values. They do not yet establish quantitative classification performance or energy savings. These should be added after a labelled evaluation and a controlled energy-use comparison.

7.5 Future Scope

Future improvements include capturing actual prototype photographs, documenting final GPIO assignments, conducting a longer labelled occupancy trial, comparing candidate classifiers on the same held-out test set, and measuring output run-time or energy consumption before and after

the control policy. WebSocket updates and additional sensors such as CO_2 may be added later, provided that the model's feature set and deployed payload are updated together. The system should remain transparent about uncertain predictions and should not use the ML output as the sole safety mechanism for critical equipment.

References

[1] A. M. Candanedo and V. Feldheim, "Accurate occupancy detection of an office room from light, temperature, humidity and CO_2 measurements using statistical learning models," *Energy and Buildings*, vol. 112, pp. 28–39, 2016, doi: 10.1016/j.enbuild.2015.11.071.

[2] "Occupancy Estimation Using Thermal Imaging Sensors and Machine Learning Algorithms," *IEEE Sensors Journal*, vol. 21, no. 6, pp. 8627–8638, Mar. 2021, doi: 10.1109/JSEN.2021.3049311.

[3] "Occupancy estimation with environmental sensors: The possibilities and limitations," *Energy and Built Environment*, vol. 6, no. 1, pp. 96–108, Feb. 2025, doi: 10.1016/j.enbenv.2023.09.003.

[4] A. Shokrollahi, J. A. Persson, R. Malekian, A. Sarkheyli-Hägele, and F. Karlsson, "Passive infrared sensor-based occupancy monitoring in smart buildings: A review of methodologies and machine learning approaches," *Sensors*, vol. 24, no. 5, Art. no. 1533, 2024, doi: 10.3390/s24051533.

[5] P. Chaudhari, Y. Xiao, M. M.-C. Cheng, and T. Li, "Fundamentals, algorithms, and technologies of occupancy detection for smart buildings using IoT sensors," *Sensors*, vol. 24, no. 7, Art. no. 2123, 2024, doi: 10.3390/s24072123.

[6] G. Cretu *et al.*, "Modeling and prediction of occupancy in buildings based on sensor data using deep learning methods," *IEEE Access*, vol. 12, pp. 102994–103003, Jul. 2024, doi: 10.1109/ACCESS.2024.3432584.

[7] H. Elkhoukhi *et al.*, "A review of occupancy sensing technologies and approaches in smart buildings," *Journal of Reliable Intelligent Environments*, vol. 15, no. 1, 2025, doi: 10.3233/RFT-240006.

[8] "Impact of occupancy behavior on building energy efficiency: What's next in detection and monitoring technologies?" *Next Energy*, vol. 8, Art. no. 100350, Jul. 2025, doi: 10.1016/j.nxener.2025.100350.

[9] A. Shokrollahi, F. Karlsson, R. Malekian, J. A. Persson, and A. Sarkheyli-Hägele, "Non-invasive occupancy estimation and space utilization in smart buildings: Leveraging machine learning with PIR sensors and booking data," *Internet of Things*, vol. 34, Art. no. 101777, Nov. 2025, doi: 10.1016/j.iot.2025.101777.

[10] A. Jafarian Amiri, K. Mahmoodi, A. Tootchi, and A. Razban, "A review of occupancy detection techniques for HVAC control: Advances and practical challenges," *Journal of Building Engineering*, vol. 113, Art. no. 113962, Nov. 2025.

[11] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[12] L. Candanedo, "Occupancy Detection," UCI Machine Learning Repository, 2016. [Online]. Available: https://archive.ics.uci.edu/dataset/357/occupancy+detection. doi: 10.24432/C5X01N. Accessed: Oct. 26, 2026.

[13] Rajiv Electronics, "ESP32 DevKitC 32D Development Board With Micro USB Connector," online listing, price INR 460.20 including GST. [Online]. Available: https://rajivelectronics.com/product/esp32-devkitc-32d-with-micro-connector. Accessed: Oct. 26, 2026.

[14] Rajiv Electronics, "PIR Motion Detector Sensor Module HC-SR501," online listing, price INR 69.60 including GST. [Online]. Available: https://rajivelectronics.com/product/pir-motion-detector-sensor-module-hc-sr501-passive-infrared-motion-sensor-module. Accessed: Oct. 26, 2026.

[15] Rajiv Electronics, "DHT22 AM2302 Digital Temperature and Humidity Sensor," online listing, price INR 118.00 including GST. [Online]. Available: https://rajivelectronics.com/product/dht22-am2302-digital-temperature-and-humidity-sensor. Accessed: Oct. 26, 2026.

[16] TecnoMate, "Photoresistor LDR Sensor Module," online component catalogue, listed price INR 61. [Online]. Available: https://www.tecnomate.in/project/esp32-smart-home-automation-iot-kit-with-relay-oled-dht11-pir-sensor. Accessed: Oct. 26, 2026.

[17] Rajiv Electronics, "5mm 3-Leg Bi-Colour LED," online listing, price INR 2.40 including GST. The table uses a rounded allowance for the LED and a resistor, not a vendor-quoted combined price. [Online]. Available: https://rajivelectronics.com/product/5mm-3-leg-bi-colour-c-a-everlight. Accessed: Oct. 26, 2026.

[18] CaresTech India, "Breadboard 830pt," online component listing, price INR 89. [Online]. Available: https://caresstech.in/. Accessed: Oct. 26, 2026.

[19] CaresTech India, “Jumper Wires 40pc,” online component listing, price INR 59. [Online]. Available: https://caresstech.in/. Accessed: Oct. 26, 2026.

[20] TecnoMate, “Micro USB Cable,” listed in its component catalogue at INR 93. This is a reference listing; confirm that the cable is a data cable and matches the board connector before purchase. [Online]. Available: https://www.tecnomate.in/project/esp32-smart-home-automation-iot-kit-with-relay-oled-dht11-pir-sensor. Accessed: Oct. 26, 2026.

APPENDIX A

API AND DATA FORMAT REFERENCE

A.1 Proposed Endpoints

- `POST /sensor-data`: accepts a sensor payload, validates it, and stores the reading.
- `POST /predict`: accepts model-compatible features and returns the occupancy label.
- `GET /latest`: returns the latest available reading and prediction.
- `GET /history`: returns recent readings and predictions for the dashboard.
- `GET /energy`: returns recent output state changes and their reasons.
- `GET /health`: returns a basic service health response.

A.2 Example Prediction Response

```
{
  "prediction": "OCCUPIED",
  "probability": null,
  "model_version": "baseline-v1",
  "timestamp": "2026-01-01T10:15:01Z"
}
```

The label and timestamp above are illustrative schema values only. The probability field should be populated only if the selected model provides a calibrated or otherwise clearly interpreted score.

A.3 Energy Policy Pseudocode

```
if prediction == OCCUPIED:
    cancel_unoccupied_timer()
    set_demo_output(ON)
else:
    if unoccupied_duration >= configured_delay:
        set_demo_output(OFF)
record_state_change_if_needed()
```

The implementation should define behavior for missing predictions, stale readings, and sensor faults. The output state should not be changed repeatedly when it is already in the requested state.

APPENDIX B

DATASET AND MACHINE-LEARNING CHECKLIST

B.1 Dataset Preparation

- Record the dataset name, version, source URL, license, and target column.
- Inspect class balance, missing values, duplicate rows, timestamps, and measurement units.
- Document every input feature and whether a corresponding physical sensor exists.
- Exclude unavailable features or install and validate the corresponding sensors.
- Split data before fitting scalers, imputers, or feature-selection steps.

B.2 Validation Rules

- Prefer a time-based or session/group-based split when observations are temporally related.
- Report the exact train/test strategy and random seed.
- Compare baseline models on the same split and feature set.
- Include precision, recall, F1-score, and a confusion matrix, not accuracy alone.
- Check false-empty errors carefully because they can cause premature output shutoff.

B.3 Deployment Checklist

- Save the fitted preprocessing and classifier pipeline together.
- Store and verify the expected feature names and order.
- Test the model with a sample payload through the API.
- Reject missing, non-finite, or implausible input values.
- Record the model version with each prediction.

APPENDIX C

FINAL VALIDATION CHECKLIST

C.1 Hardware and Sensor Readiness

ESP32 firmware reads each installed sensor.

Sensor values and units are verified and documented.

LDR divider voltage remains within the ESP32 ADC input range.

Power, ground, and signal connections are checked.

Demonstration output is low voltage and properly driven.

C.2 ML and Backend Readiness

Dataset source and feature list are documented.

Evaluation split and metrics are reproducible.

Deployed features match the actual hardware.

API validates payloads and handles failures.

Database stores readings, predictions, and energy events.

C.3 Dashboard and Energy Logic Readiness

Dashboard displays the latest values and stale-data status.

Historical charts use labelled units and timestamps.

The unoccupied delay is configurable and tested.

A renewed occupied prediction cancels the switch-off timer.

Fault cases and manual override behavior are documented.

C.4 Results to Attach

Model comparison table with actual measured values.

Confusion matrix and classification report.

Photographs of the completed hardware prototype.

Dashboard screenshots.

API and integration test evidence.

Actual bill of materials and project timeline.